GEN AI WITH LLMS · TOPIC 1

Text Processing Fundamentals

From Tokens & Counts to Learned Word Embeddings

Laura Wynter, PhD
SMU School of Computing and Information Sciences
IS469 · AY2026–27

ROADMAP

What we'll cover

01 The course: goals & structure

02 Text normalization & tokenization

03 n-gram language models

04 Embeddings: BOW & one-hot

05 TF-IDF & cosine similarity

06 Typical uses: IR, classification, extraction

07 Neural network recap

08 Word2Vec: CBOW & skip-gram

09 Bridge to modern embedding models

10 Hands-on lab

FRAMING

Overview of the course

- Text processing fundamentals (today) → Transformer architecture & the main LLMs
- Customizing & fine-tuning LLMs → evaluating LLMs → deploying LLMs
- Uses of LLMs: retrieval / RAG (T7), and agentic flows & Agent Skills (T8)
- Advanced LLM architectures, and the ethics of LLMs & latent reasoning
- Format: combined lecture and in-class tutorial / lab each week

FRAMING

Learning outcomes for this lecture

- Understand the goals and structure of the course
- Explain early text processing & early text generation (n-gram models)
- Understand, explain and use embeddings, BOW and TF-IDF
- Explain and compute cosine similarity
- Refresh neural networks for text (say so if this is new to you)
- Understand and use Word2Vec, an early NN-based embedding model

FRAMING

Why process text at all?

sentence not a good structure:
- not of fix length (size)
- may lose meaning in a sentence (length)

- Text is unstructured data, we transform it into a form ML can use
- Vectors are the most suitable representation for machine learning
- Uses: text classification, text generation (predict next word), translation, dialogue
- Challenges: ambiguity & context-dependent meaning; diversity of output; long-range dependencies

01

Text Normalization & Tokenization

Before a model sees text, we clean it and split it into units it can count.

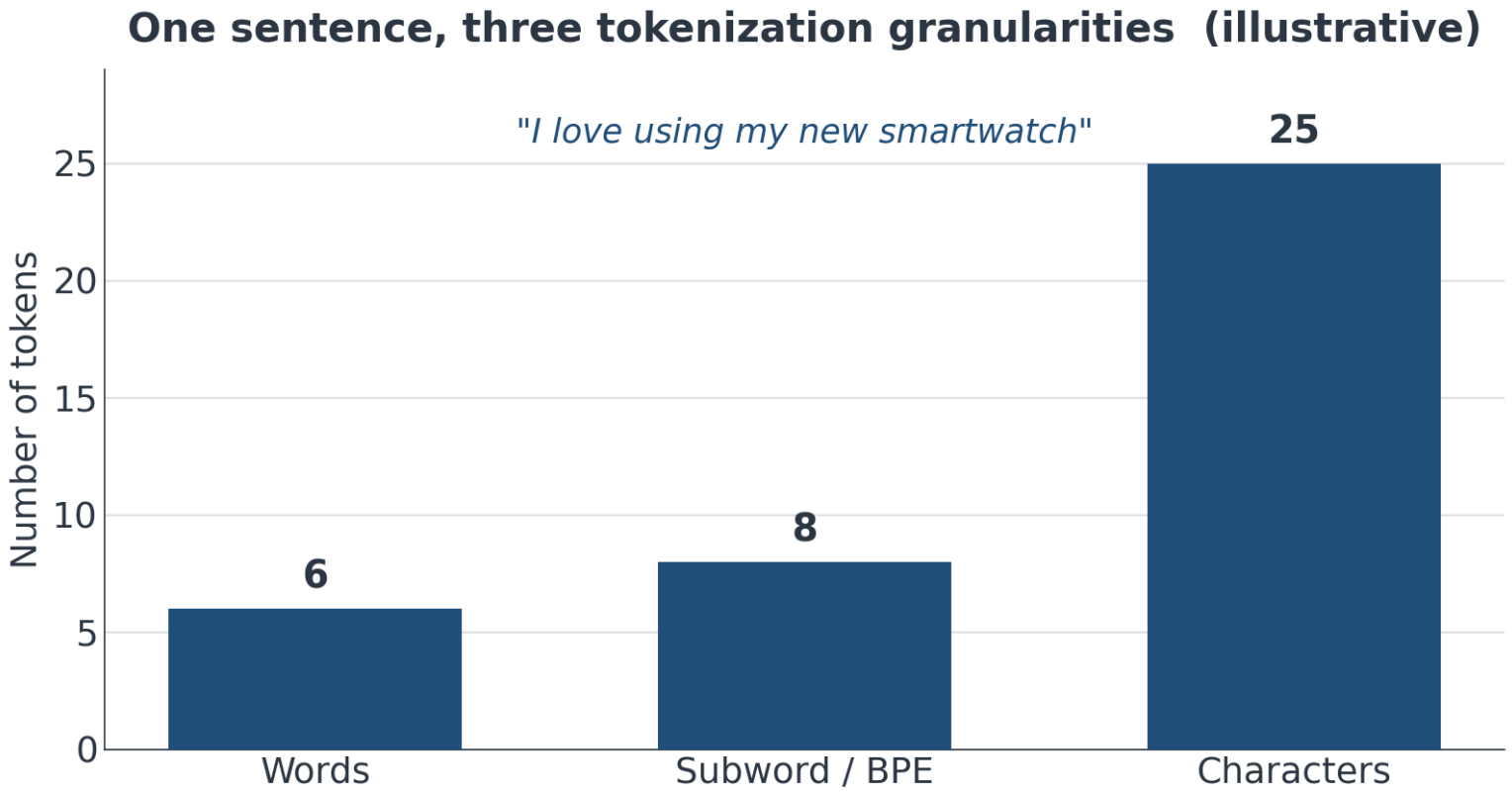
Main tasks in early text processing

giving text more structure

lemmatization: converting word to root form
stopword removal: removing words of not much meaning

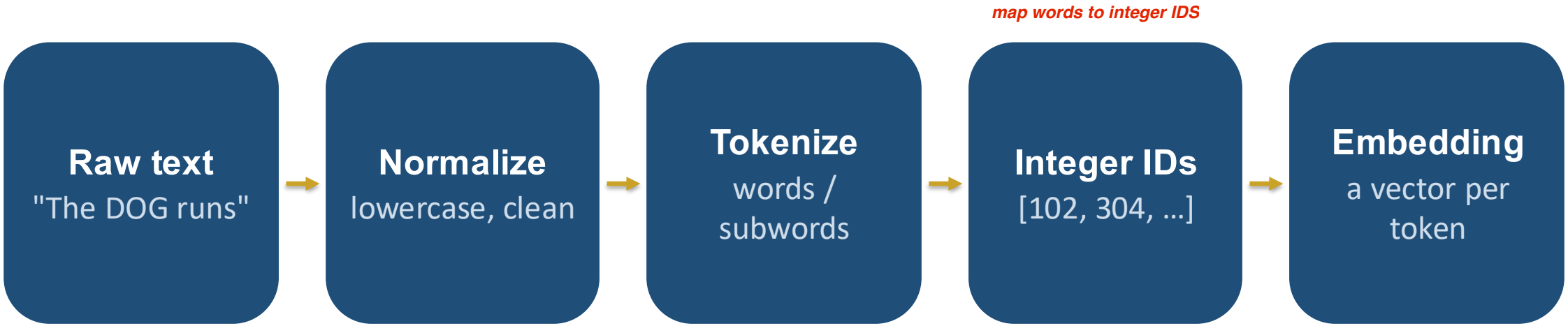
- Text normalization: lowercasing, punctuation handling, whitespace stripping, stopword removal, lemmatization
- Tokenization: split text into units, word-level, subword-level (BPE, WordPiece), or character-level
- Out-of-vocabulary (OOV): at run time a word appears that was never seen in training
- Normalization mattered most when models and compute were far less powerful than today

Granularity sets how many tokens you get



The same sentence becomes very different token counts depending on the level you split at.

The text-processing pipeline



Every step turns messy text into numbers a model can compute on.

Normalization, lowercasing

reducing dimensionality of the problem by reducing number of variations of words

- Convert text to lowercase so surface variants collapse to one form
- "Dog", "dog" and "DOG" are all treated as the same word
- Before: "The Cat chased the Dog. I met a DOG on the street."
- After: "the cat chased the dog. i met a dog on the street."
- Trade-off: cheap and consistent, but discards case that can carry meaning (e.g. Apple vs apple)

Normalization, punctuation & lemmatization

- Punctuation removal, before: "the cat chased the dog. i have two dogs."
  - after: "the cat chased the dog i have two dogs"
- Modern practice: keep punctuation, it carries sentiment and marks sentence boundaries
- Lemmatization maps a word to its dictionary base form (lemma) using morphology & part-of-speech
  - "meetings" → "meeting"; "better" → "good"; "was" → "be"

root form

- Stemming is cruder: it just chops suffixes, so the result need not be a real word
  - "normalization" → "normal"; "studies" → "studi"

Numbers, dates, Unicode & spelling

- Numbers and dates can be standardized or replaced with special tokens to absorb format variation
- Before: "I have 2 meetings on 21/1/2025."
- After: "I have NUM meetings on DATE."
- Unicode normalization & spelling correction keep characters consistent, "café" → "cafe"

Out-of-vocabulary words & subwords

- Simplest fix: map any unseen word to a single OOV / UNK token, but this throws away information
- Better: let the model read new words by breaking them into subwords
- Before: "I love using my new smartwatch."
- After: "I love using my new sm ##art ##watch" (## = attach to previous piece)
- Still core today: subword tokenization splits rare words into pieces likely seen elsewhere

02

n-gram Language Models

A first attempt at text generation: predict the next word from the last few words.

n-gram models: a first attempt at generation

today: given history of what was generated/ user query history —> we generate text

- Attributed to Claude Shannon (1948), same goal as a modern LLM, very different mechanics
- Predict the probability of a word given its history h: P(w | h)
- An n-gram is an n-word sequence; with n = 2 we use 2-word sequences called bi-grams
- Generate text by predicting the next word repeatedly, until an end-of-text marker

Scoring a sentence with n-grams

- Example sentence: "The sky is blue"
- Uni-gram: p(sentence) ≈ p(The) · p(sky) · p(is) · p(blue)
- Bi-gram: mark sentence start/end with <s>, condition each word on the previous one

  history being start of sentence

  · p(<s> The sky is blue </s>) ≈ p(The|<s>) · p(sky|The) · p(is|sky) · p(blue|is) · p(</s>|blue)
- A tri-gram conditions on the previous two words, more context, more parameters

Bi-grams of "i want to eat chinese food"

MODEL LIMITATIONS

1. such model gives repetitive generation

2. output is entirely limited to corpus vocab e.g won't output chinese noodles)

bi-gram	C(prev, w)	C(prev)	P(w | prev)
	count of prev being <s> and first word is i	*count or prev being <s>*	
<s> → i	250	1,000	0.25
i → want	720	2,400	0.30
want → to	520	800	0.65
to → eat	680	2,600	0.26
eat → chinese	15	750	0.02
chinese → food	78	150	0.52

probability of like is less than 0.30

given history of I want, the next word is 65% probability

2% is the highest probability, there are a lot of other options

P(w | prev) = C(prev, w) / C(prev). Illustrative counts (Berkeley Restaurant corpus; exact figures in Jurafsky & Martin, SLP3 ch. 3).

Scoring the sentence, bi-gram by bi-gram

- Chain the bi-gram probabilities for "i want to eat chinese food":
  - $P = P(i|<s>) \cdot P(want|i) \cdot P(to|want) \cdot P(eat|to) \cdot P(chinese|eat) \cdot P(food|chinese)$
  - $= 0.25 \cdot 0.30 \cdot 0.65 \cdot 0.26 \cdot 0.02 \cdot 0.52 \approx 1.3 \times 10^{-4}$

at each step chooing the highest probability

- A tiny number is expected, any one specific sentence is rare, but the model still ranks fluent word orders above scrambled ones
- The point: probabilities come straight from corpus counts (C(prev, word) / C(prev)), no learning of meaning yet
- Full Berkeley Restaurant tables: Jurafsky & Martin, SLP3 ch. 3 (web.stanford.edu/~jurafsky/slp3)

Why n-grams fell out of favor

- Only a fixed, short context is used; larger n helps but the counts explode
- Probabilities simply encode facts about the training corpus
- Any word/pair unseen in training gets probability zero → brittle, no creativity
- We'd rather learn relations between words: "I have a pet cat" → cat ~ dog → "I have a pet dog"
- So generation took a back seat, but as the web & search grew, text representation became central

03

Embeddings, Bag-of-Words

Represent meaning as a vector, so we can measure how close two texts are.

Word meaning as an embedding

idea is you want to search for something (e.g Google searches closest thing to query given embedding)

- Define a word's meaning by its distribution, its neighboring words and grammatical context
- Shift from text generation to text representation, to capture meaning

vector is an arrow with magnitude and direction, represent in 3 dimension

- The vector is called an embedding because the word is embedded into a space
- Distance in that space defines similarity between words or sentences

represent text in vectors so we can do operations —> distance —> we care about distance because of search

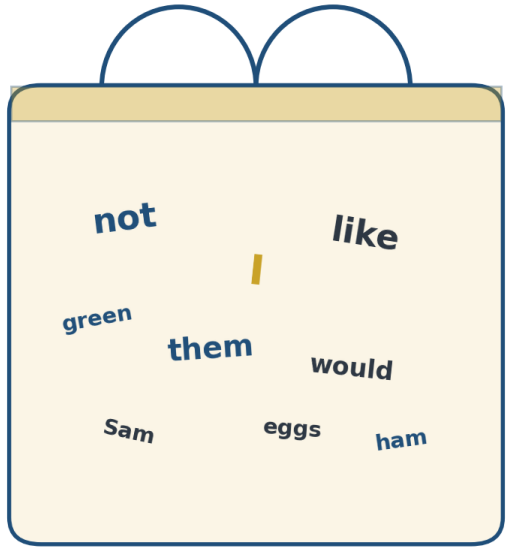
type in SMU —> google returns output of those with different distances

Bag-of-words (BOW)

- Represent text as a set of words using the frequency of each word
- BOW gives an embedding via one-hot / count encoding, a slot per vocabulary word
- So the BOW vector length = the number of words in the whole vocabulary
- Disadvantages: very high-dimensional, sparse, ignores word order, and no notion of similar meaning

A bag of words: order thrown away

Bag-of-words: throw the words in a bag, keep only the counts



Word order is lost; larger word = more frequent (I appears 5×)

need one hot encoding

BOW keeps only which words appear and how often, not their order. "I do not like them" and "them like not do I" give exactly the same bag.

BOW worked example, "Green Eggs and Ham"

set of unique word = vocabulary

non repeated

counting frequency of each words then use one hot encoding to convert it into a vector

- The book has 50 unique words → embed each page as a 50-dim vector
- Page text: "I would not like them here or there. I would not like them anywhere. I do not like green eggs and ham. I do not like them, Sam I am."
- Count each vocabulary term to get the page's vector, e.g. I = 5, not = 4, like = 4, them = 3
- Simple and interpretable, still useful for teaching how text becomes numbers

A page becomes a count vector

term	I	would	not	like	them	green	eggs	ham
count	5	2	4	4	3	1	1	1

A vector of length 50 (size of vovcab)

one hot encoding will require an order, embedding vector is in alphabetical order (INVARIANT AND ALWAYS THE SAME)

similar page will have similar frequency of words

a lot of noise as common word is dominated

"Green Eggs and Ham": each page → a 50-dim count vector (excerpt shown). Order is lost.

04

TF-IDF & Cosine Similarity

Down-weight common words, then measure the angle between vectors.

From raw counts to TF-IDF

Term Frequency (BOW)
Inverse Document Frequency (downweight most frequent words)

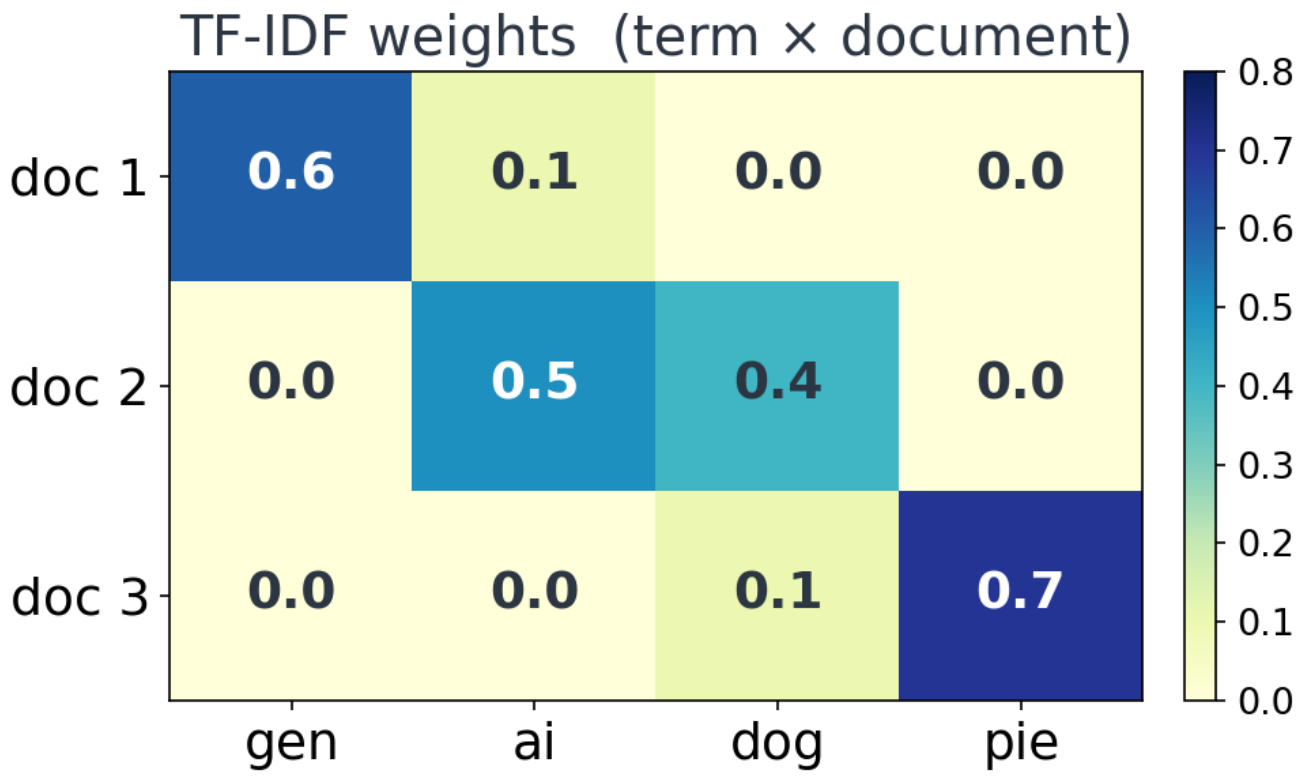
e.g any documents/page may have I, not , like . if every page have I then we give it a lower score

- Plain BOW counts are dominated by the most common words, which carry little information
- TF-IDF = term frequency × inverse document frequency, same vector length, reweighted values
- IDF down-weights a term by how many documents it appears in, common terms count for less
- Especially useful in information retrieval (search) and text mining

TF-IDF, precisely

- Term frequency: TF(i, d) = count(i, d) / (total words in d) [share of word i in document d]
- Inverse document frequency: IDF(i, D) = log(NumDocs / |{d : i ∈ d}|)
- Combine: TF-IDF(i, d) = TF(i, d) · IDF(i, D)
- A word frequent in one document but rare across the corpus scores highest
- Result: distinctive terms rise, boilerplate words sink, better retrieval signal

TF-IDF re-weights the count matrix



Distinctive terms (frequent in one document, rare across the corpus) get the highest weight.

comparison between the two vectors

Cosine similarity

we want to do computations on the embedding

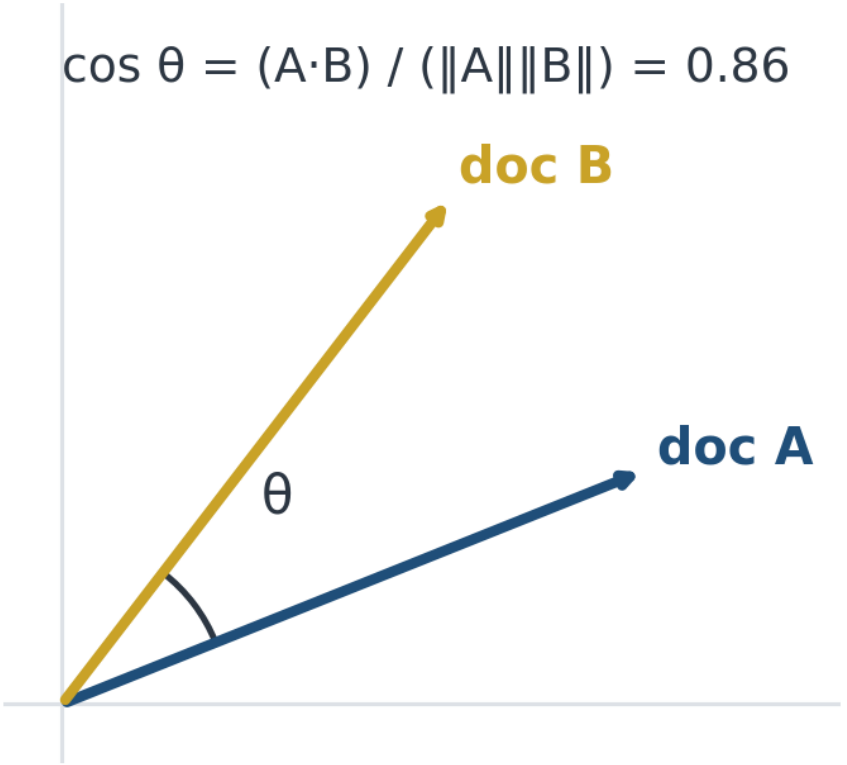
- We often want the document most similar to a user's query (a search engine's core task)
- Two vectors are similar when they have a large dot (scalar) product
- But the dot product grows with vector length, longer counts inflate the score
- Cosine similarity is the normalized dot product: it measures the angle, not the magnitude

Cosine similarity, worked

computation between vector of query and each page

- Dot product: $a \cdot b = |a|\ |b| \cos\theta$ *dot product scaled by length of a and b (normalised dot product)*
- So $\cos\theta = (a \cdot b) / (|a|\ |b|) = \Sigma_i v_i w_i / (\sqrt{(\Sigma_i v_i^2)} \cdot \sqrt{(\Sigma_i w_i^2)})$
- Range 0–1 for non-negative count/TF-IDF vectors; 1 = same direction, 0 = no shared terms
- Example: query "green eggs" vs each page, rank pages by cosine score, highest first
- Length normalization is why a short query can still match a long relevant document

Cosine similarity is the angle, not the length



Two documents are similar when the angle between their vectors is small, regardless of length.

Typical uses of BOW / TF-IDF + cosine

- Quantify how similar two documents are, or a document and a user query
- Search / information retrieval: rank results by cosine score, more relevant scores higher
- Text classification
- Key information extraction

§4 · IN CLASS

Corpus-building activity

> *"We'll build a small corpus together, encode it with BOW, watch frequent words dominate, then watch TF-IDF down-weight them."*

In-class activity · reinforces why raw counts mislead and IDF helps

LAB 1

Hands-on Lab 1, BOW / TF-IDF / cosine

- Topic1.ipynb: build BOW and TF-IDF with scikit-learn, then rank documents by cosine similarity
- See highly frequent words dominate raw counts, and watch TF-IDF down-weight them
- This lab practises exactly what we just covered (§3, §4), do it now before moving on

05

Neural Network Recap

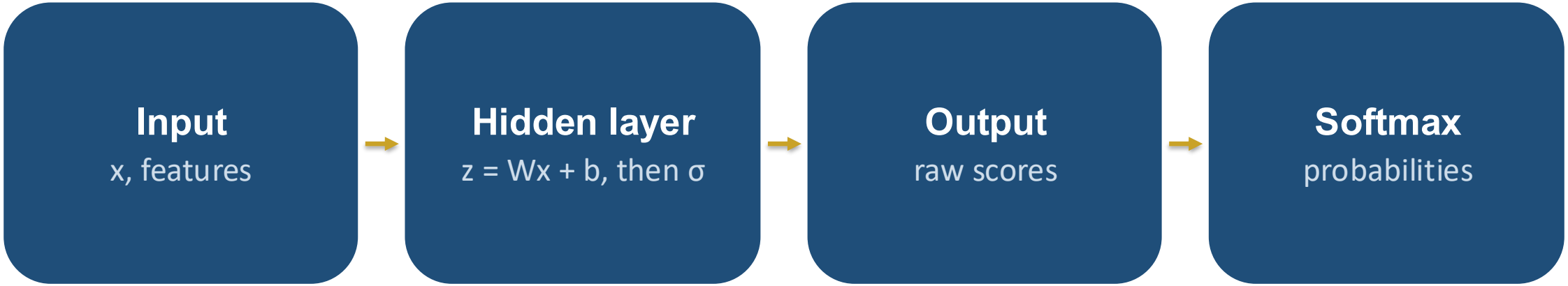
A quick refresher, because Word2Vec, and every LLM, is built from these.

A neuron in a neural network

BOW is too computationally expensive & loses order

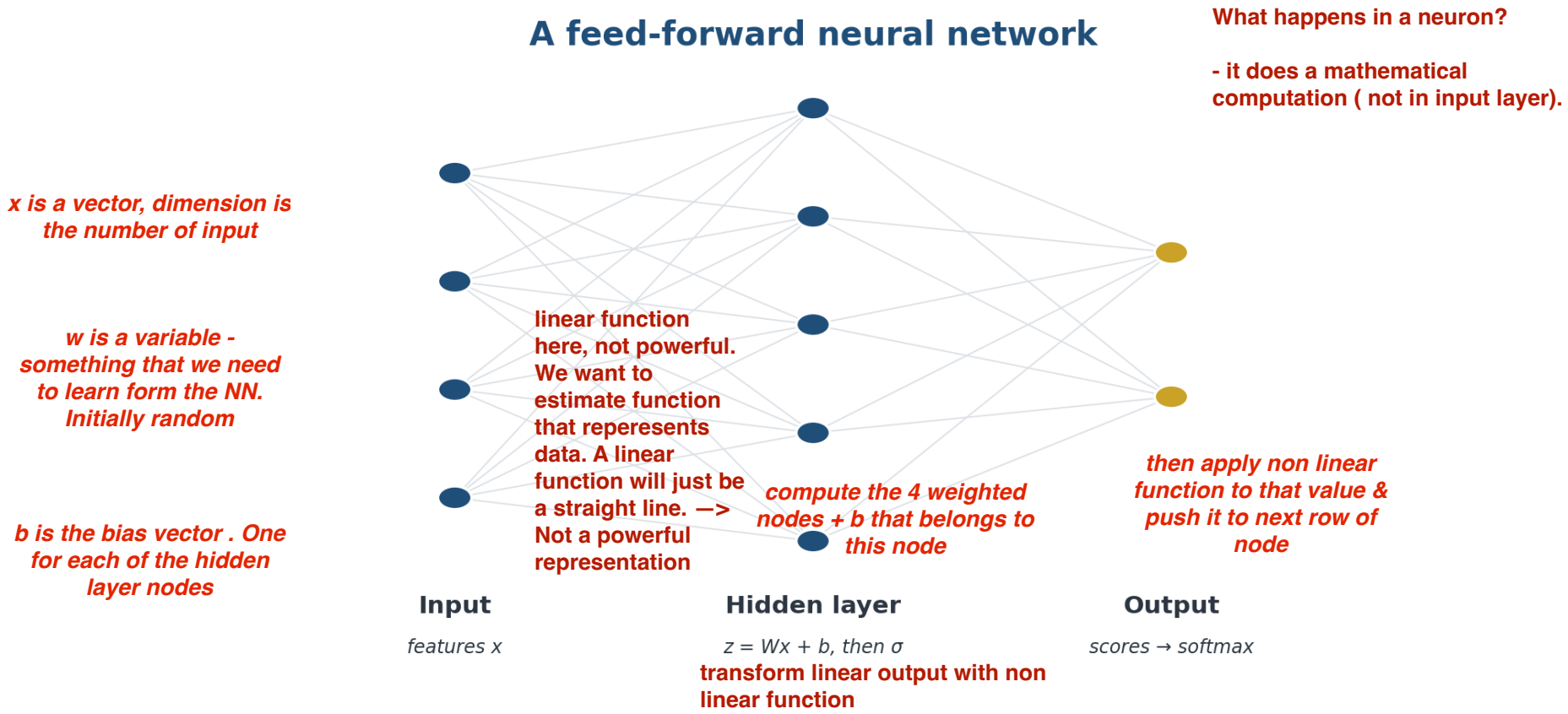
- A neuron computes $z = b + w \cdot x$, then applies a nonlinear activation $y = \sigma(z)$
- Common activations: sigmoid, tanh, ReLU, GeLU
- ReLU: $\sigma(z) = \max(0, z)$, cheap and the default in many networks
- Softmax generalizes sigmoid: $e^{z_i} / \Sigma_k e^{z_k}$, used as the last layer to output a probability distribution

A feed-forward network, end to end



Word2Vec, and every LLM, is built from exactly these pieces.

A feed-forward neural network



Input features → a hidden layer (Wx + b, then a non-linearity) → output scores → softmax. The building block of everything that follows.

§5 · DEPTH

One-layer vs. two-layer networks

One-layer network

- Simplest neural network
- Replicates binary or multi-class logistic regression
- Linear decision boundary (plus the output activation)
- No hidden representation

Two-layer network

- Adds a hidden layer of nonlinearity
- Learns intermediate representations
- Can capture more complex phenomena
- The basis for the Word2Vec model next

Feed-forward networks (FFN / MLP)

- Stack fully-connected layers → a Feed-Forward Network (FFN)
- Also called a fully-connected network (FCN) or multi-layer perceptron (MLP)
- Power comes from learning features from data, not hand-engineering them
- We'll meet FFNs again inside the transformer, they are a core layer type there

no jumping of layers

06

Word2Vec

Learn dense word vectors with a shallow neural net instead of raw counts.

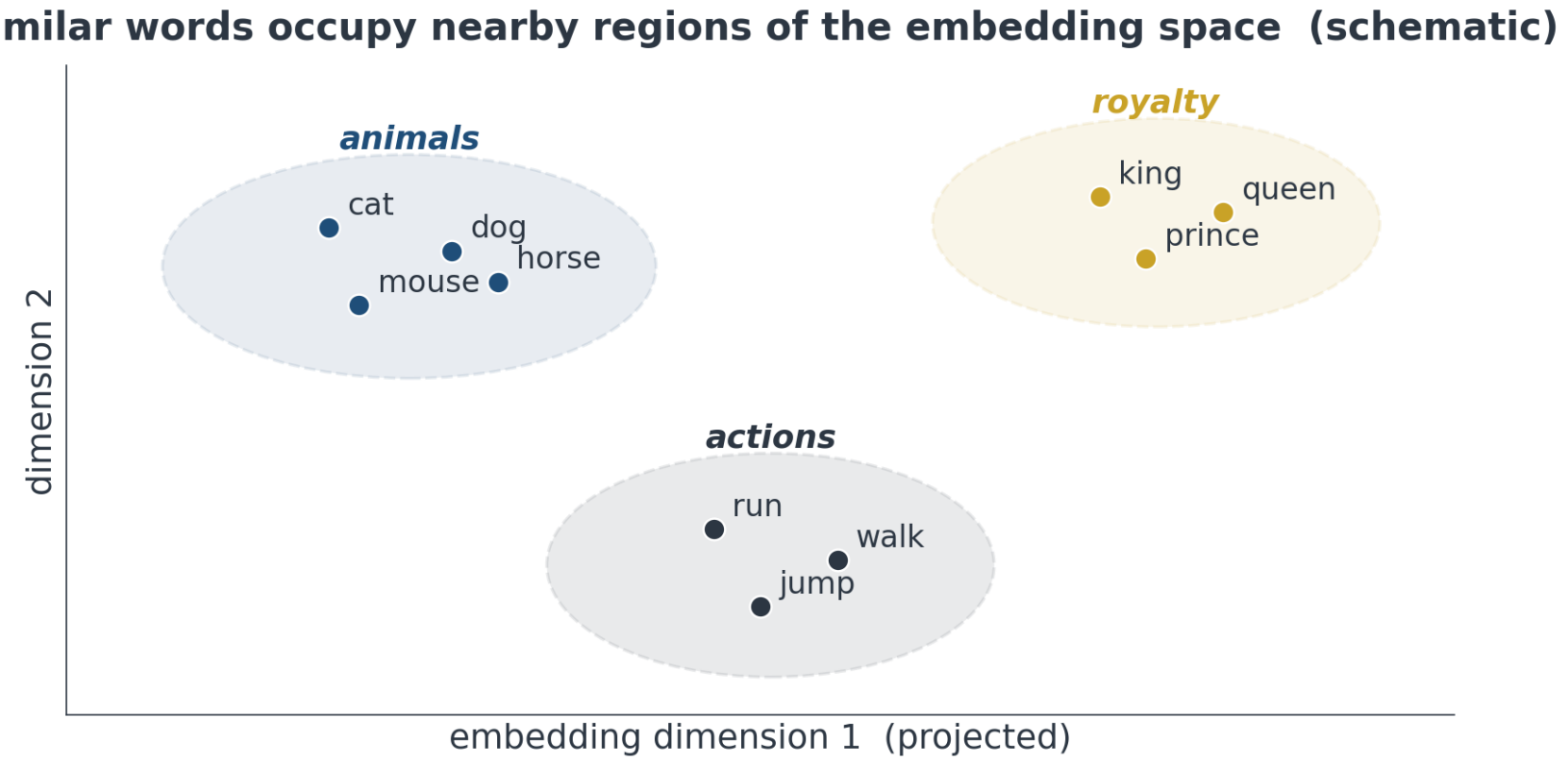
Word2Vec: learn embeddings, don't count

using weights on Neural network as our embedding vector

when train NN we are computing w & b , we train by pumping text through the neural network and ask it to predict something (means it is looking at word order)

- A completely new idea when introduced in 2013
- Train a classifier to predict a word from (or against) its context
- Use a shallow 2-layer NN: input layer, one hidden layer, output
- The learned hidden-layer weights become the embedding vector for each word
- Vector arithmetic works: vec(king) − vec(man) + vec(woman) ≈ vec(queen)

Learned embeddings cluster similar words



A schematic 2D projection of Word2Vec-style embeddings: words with similar meaning land near one another, so distance measures similarity, exactly what BOW cannot do.

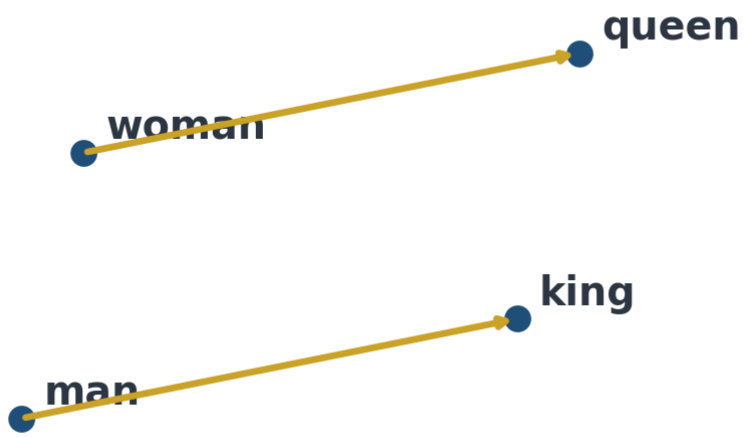
Vector arithmetic captures meaning

still uses cosine similarity

- Similar contexts → nearby vectors
- Relationships become directions in the space
  - gender, royalty, tense each become a direction
- So analogies turn into arithmetic:
  - king − man + woman ≈ queen

understanding rs between words

king − man + woman ≈ queen



The man→king and woman→queen arrows are parallel.

size of emeddingis length of number of hidden (middle nodes)
u set that and value is the weight

Two ways to train Word2Vec

continuous
CBOW

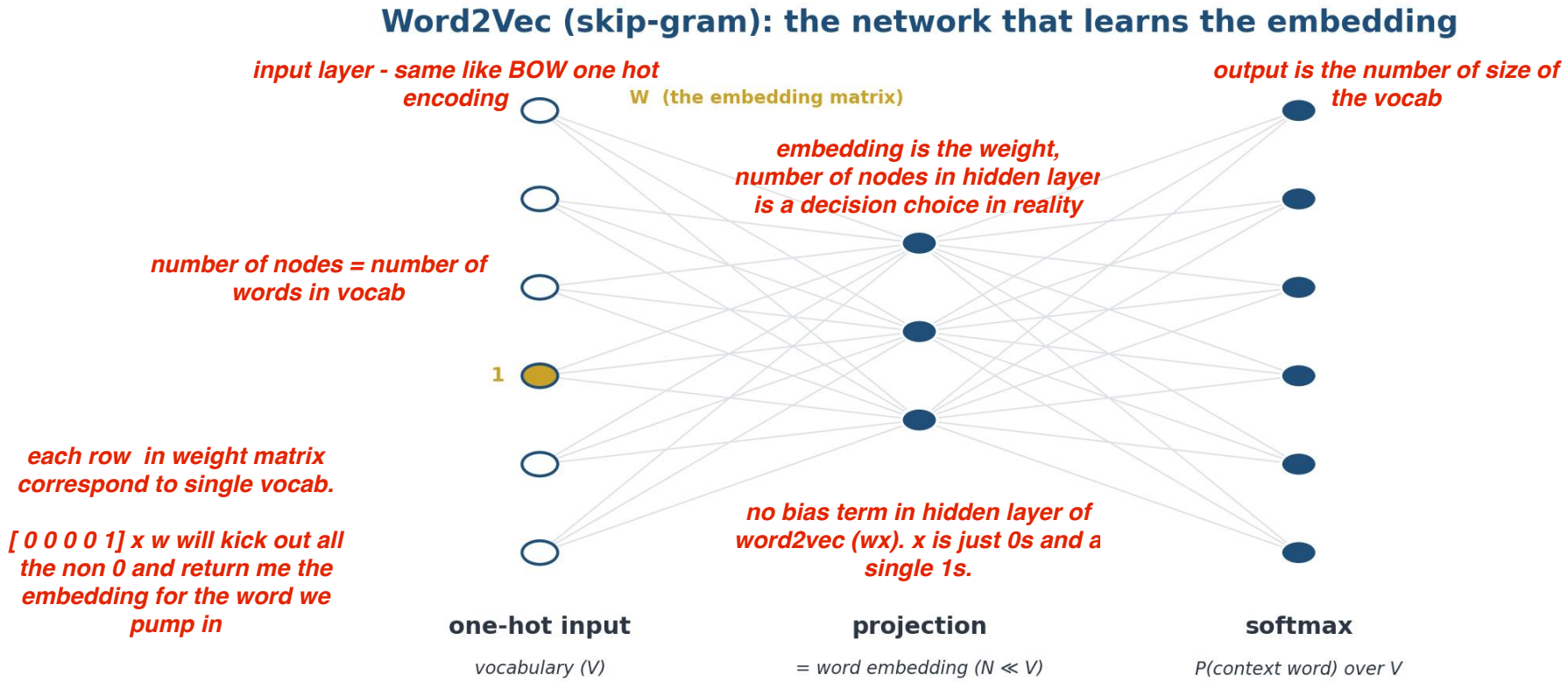
pump text in NN and train it to predict the missing word

- Predicts the middle word from its context
- "SMU is the ? university in Singapore!"
- Word order has no effect
- Faster; good for frequent words

Skip-gram

- Predicts the surrounding words from the middle one
- "? ? ? best ? ? ?"
- Uses a window before and after the current word
- Better for rare words and larger corpora

The Word2Vec network (skip-gram)



One-hot input word → a small projection layer (the learned embedding, N ≪ V) → softmax over the vocabulary predicting context words. The hidden layer IS the word vector.

Word2Vec vs. BOW / TF-IDF

BOW / TF-IDF

- Sparse, one slot per vocabulary word
- Dimension = |V| (can be 10,000+)
- No notion of relatedness between words
- Built from counts

Word2Vec

- Dense, low-dimensional (e.g. 300)
- Similar contexts → nearby vectors
- Captures relations (king−man+woman≈queen)
- Learned by a neural network

Limitations of Word2Vec

- One fixed vector per word, no context sensitivity ("bank" river vs. money)
- Unknown words get no meaningful vector (often a random one)
- No sub-word sharing, so morphologically related words are independent
- Hard to scale across languages, no parameter sharing for cross-lingual use

07

From Word2Vec to Modern Embeddings

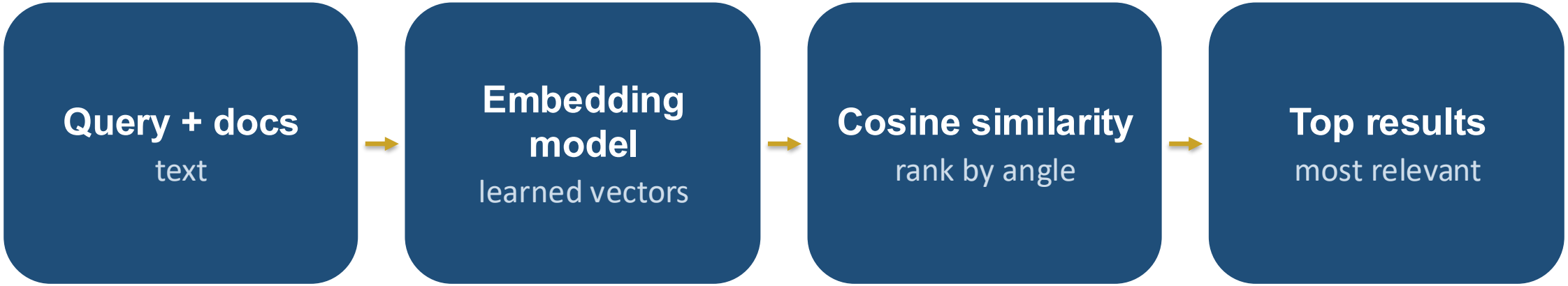
Static word vectors gave way to contextual, sentence-level embedding models.

§7 · BRIDGE

Word2Vec → modern embedding models

- Word2Vec is static: one vector per word, no context, modern models fix exactly that
- Contextual embeddings (BERT and successors) give a word a different vector per sentence
- Sentence / passage models embed whole texts, 2026 MTEB leaders: Gemini Embedding, Qwen3-Embedding (top open-weight), Cohere embed-v4, BGE-M3, voyage-3.5
- These power dense retrieval, encode query & documents, rank by cosine, just like today but with learned meaning
- Forward link: this is the retrieval backbone of RAG (Topic 7)

How modern embeddings power search (RAG)



Same cosine machinery as §4, but with learned, contextual vectors. This is the RAG backbone (Topic 7).

Key takeaways

01

Text → vectors

Normalize and tokenize, then represent text as vectors ML can work with.

02

Counts have limits

n-grams and BOW/TF-IDF are simple and interpretable but sparse, order-blind, and no sense of meaning.

03

Cosine similarity

The normalized dot product measures the angle between vectors, the core of retrieval.

04

Learned embeddings

Word2Vec learns dense vectors that capture relations; modern models make them contextual and power RAG.

LAB 2

Hands-on Lab 2, Word2Vec

- Topic1-w2v.ipynb: train Word2Vec with gensim on the 20-newsgroups corpus
- Explore the vector space: nearest neighbors and king − man + woman ≈ queen
- Come away able to explain, compute and use each representation, the foundation for the whole course